

Estado del CRM

Cómo está conectado y cómo funciona

Mapa técnico completo del sistema: captura de llamadas, datos, seguridad, aplicación, bot e integraciones.

Refleja el código real al **2026-06-08** — no de memoria. Generado tras mapear los 4 subsistemas.

- Next.js 15
- Supabase
- Vercel
- 800.com
- Meta Lead Ads
- Claude · Hermes

Contenido

1. Mapa general

2. Captura de llamadas (800.com)

3. Secuencia de una llamada entrante

4. Meta Lead Ads

5. Modelo de datos
6. Seguridad y acceso (RLS)

7. La aplicación (páginas)

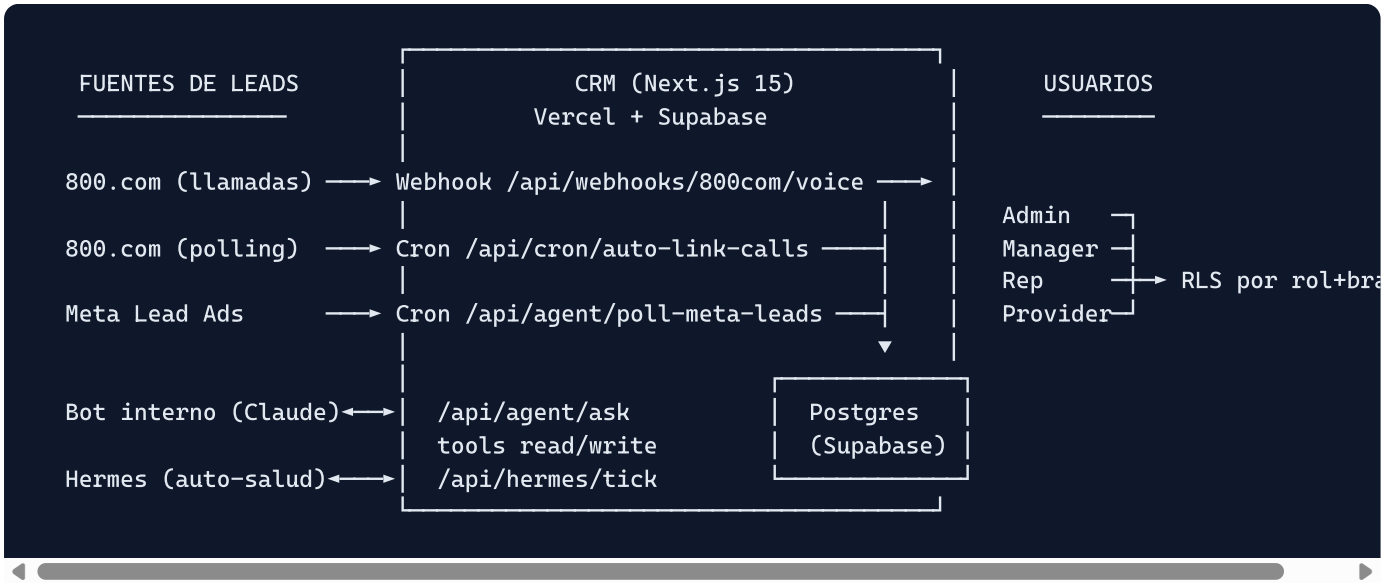
8. Bot interno y Hermes

9. Crons y disparadores

10. Procedimientos operativos

11. Salud actual y pendientes

0 · Mapa general en una frente



El CRM es un **pipeline de ventas y citas multi-marca**: **Lead → Llamada → Cita → Venta → Pagos (planes a plazos)**, con control de acceso por rol y por marca.

- Framework:** Next.js 15 App Router (Server Actions + Server Components)

- **Base de datos / Auth / Realtime:** Supabase (Postgres)
- **Hosting:** Vercel (Rolling Releases — el deploy NO auto-promueve el alias prod, hay que `vercel alias set`)
- **Integraciones vivas:** 800.com (call tracking) y Meta Lead Ads (formularios Facebook)
- **IA:** bot interno con Claude (`claude-sonnet-4-6`) + Hermes (bot de auto-salud)

1 · Captura de llamadas (800.com)

Hay 3 caminos que meten llamadas/contactos al CRM, todos deduplican y son idempotentes.

1.1 Webhook en tiempo real — `webhooks/800com/voice/route.ts`

El receptor protegido. Es lo que hace que una llamada aparezca al instante.

- **Eventos:** `call_started` (INSERT, outcome=null) · `call_completed` (UPDATE outcome+duración) · `call_decorated` (UPDATE grabación+transcript) · `call_updated` · `sms_received` ignorado.
- **Auth:** secreto en query param `?secret=` (`EIGHTHUNDRED_WEBHOOK_SECRET`).
- **Ejecución asíncrona:** responde `200` de inmediato y hace el trabajo en `after()` con try/catch. Evita el timeout corto de 800.com.
- **Dedup:** por `external_id` + `source='800com'`. Si chocan started/completed, el 2º INSERT falla por índice único y se trata como OK ("race: duplicate").
- **resolveTracking:** match por `provider_metadata.number_id` → fallback por número marcado → `{ brand_id, rep_id, tracking_number_id }`. Rep = primer rep/manager/admin activo del brand; si no, admin activo más antiguo.
- **Dirección:** `isInbound===true || direction=="inbound" || (ambos undefined)`. Si `isInbound===false` → outbound (evita el "toast fantasma").
- **Columnas del INSERT:** `brand_id, lead_id, rep_id, direction, outcome, duration_seconds, caller_e164, dialed_e164, tracking_number_id, source, external_id, recording_url, called_at, ringing_at, transcription`.

Regla operativa: no tocar este archivo sin autorización explícita. Históricamente romper aquí cortó la captura por horas.

1.2 Cron de enlace — `cron/auto-link-calls/route.ts`

Engancha llamadas huérfanas (`lead_id IS NULL`) a leads.

- **Auth:** Bearer `CRON_SECRET` o `HERMES_SECRET`. **Ventana:** `?days=N` (default 1).
- **Topes:** `ORPHANS_LIMIT=500`, `CONV_MAX_PAGES=5`, `MAX_ENRICH=50` (no exceder `maxDuration=60`).
- **Flujo:** resuelve teléfono → busca conversación (`enhancedCallerId`) → match por (`brand_id, phone`) o crea lead (`assigned_rep_id:null`, pool) → enriquece ECID → `UPDATE calls SET lead_id`.

1.3 Toast en tiempo real + identificación

`incoming-call-toast.tsx` + `calls/actions.ts` — lo que ve el rep cuando entra una llamada.

- El toast escucha `postgres_changes` (INSERT en `calls`) por Supabase Realtime.
- Si no hay lead, marca "identificando..." y llama a `identifyCaller(callId)`: (1) si ya tiene lead → nombre sin costo; (2) match por teléfono → enlaza sin costo; (3) ECID lookup (cacheado=gratis, en vivo≈\$0.05) → crea lead con nombre/dirección/carrier.
- Muestra: nombre o teléfono, estado del lead, saldo del plan activo, última cita, ciudad/estado/carrier.

1.4 El cliente de API — `lib/integrations/800com.ts`

- `GET /v2/calls` — historial (cursor, sleep 1100ms anti rate-limit 60/min).

- `GET /companies/{id}/conversations` — contactos con `enhancedCallerId` (nombre, ciudad, carrier, emails). Normaliza cursor.
- `POST /v2/companies/{company}/ecid/lookups` — Enhanced Caller ID: dirección, lineType, emails.
Cacheado=gratis; en vivo=cobra.
- `maybeEnrichLead` (`ecid-enrich.ts`): nunca lanza error, salta la API si ya hay `address_line1` , y solo llena campos vacíos.

1.5 Backfill manual (admin)

- `syncTrackingNumbersFromEightHundred` — registra los números en BD (requisito para que el webhook resuelva).
- `backfillCallsFromEightHundred` — importa llamadas históricas (7d default, hasta 200 páginas).
- `backfillLeadsForOrphanCalls` — como el cron pero ventana larga; `dryRun=true` por defecto.
- `syncContactsFromConversations` — sincroniza TODOS los contactos (+ archivados) + ECID opcional; cuenta `ecid_lookups/ecid_charged` .

2 · Secuencia exacta de una llamada entrante

Lo que pasa, en orden, desde que marcan un número de campaña hasta que el rep ve la info completa. Cada evento de 800.com es un POST separado al mismo webhook y toca la **misma fila** de `calls` (dedup por `external_id`).

CLIENTE marca +1-888-307-3743 (número "Buses" de Si Se Pierde)

```
T+0.0s 800.com → POST /api/webhooks/800com/voice?secret=...
type: "call_started"
data: { id: 998877, caller: "+17149260731",
        dialed: "+18883073743", numberId: 416682,
        isInbound: true, startedAt: "..." }
```

responde 200 { ok, accepted } AL INSTANTE
y agenda el trabajo en after() (no bloquea a 800.com)

after() [T+0.1s aprox]

1. dedup: ¿existe call external_id=998877? → NO
 2. resolveTracking(numberId 416682) → tracking "Buses"
→ { brand_id: SiSePierde, rep_id:, tracking_number_id }
 3. dirección: isInbound === true → "inbound"
 4. findLeadByPhone → NULL (cliente nuevo)
 5. INSERT calls { external_id, source: "800com", direction: "inbound",
outcome: NULL, ← "sonando"
caller_e164, dialed_e164, tracking_number_id,
brand_id, rep_id, lead_id: NULL, called_at, ringing_at }
- ▼ el INSERT dispara Supabase Realtime

```
T+0.3s NAVEGADOR del rep - incoming-call-toast.tsx
recibe el INSERT → muestra TOAST + 🔔
lead_id=NULL → "identificando..." + identifyCaller(998877)
```

- ▼ identifyCaller (mientras suena)
- a. ¿la call ya tiene lead? NO
 - b. match por teléfono → NO
 - c. lookupEnhancedCallerId(...) ← ECID (cacheado gratis / vivo ≈\$0.05)
→ { name: "Brian Franks", city, state, street, zip, carrier }
 - d. crea LEAD (status: new, source: inbound_call, assigned_rep_id: NULL)
 - e. UPDATE calls SET lead_id =

EL TOAST se actualiza: "Brian Franks · Phoenix, AZ · Verizon"
(el rep contesta con nombre y datos en pantalla)

———— la llamada termina ————

```
T+45s POST ... type: "call_completed"
data: { id: 998877, result: "answered", duration: 42, endedAt }
```

after(): dedup encuentra la fila → UPDATE (no INSERT)
outcome "answered" → "connected"; duration_seconds=42

```
T+90s POST ... type: "call_decorated"
data: { id: 998877, recordingUrl: "https://...", transcription }
```

after(): UPDATE misma fila → recording_url, transcription

/calls y /calls/[id]: Brian Franks · inbound · connected · 0:42 ·
campaña "Buses" · grabación reproducible · transcript

Puntos clave

- **Una sola fila, varios eventos.** started la crea; completed y decorated la actualizan por `external_id` . Nunca se duplica.
- **El toast no espera al cron.** La identificación ocurre vía `identifyCaller` mientras suena.
- **Atribución desde el primer evento.** `dialed_e164` + `tracking_number_id` se escriben en el INSERT.
- **Si el ECID en vivo falla,** la llamada queda capturada con `caller_e164` ; el cron reintenta después.
- **Red de seguridad:** `poll-800com` (diario) y `backfillCallsFromEightHundred` (manual) recuperan por `external_id` .
- **Saliente:** idéntico pero `direction="outbound"` y el toast no se dispara.

3 · Meta Lead Ads (formularios de Facebook)

- **Módulo:** `lib/integrations/meta-lead-ads.ts`. **Polling:** `api/agent/poll-meta-leads` (cron-job.org cada 15 min + cron Vercel diario).
- **Flujo:** carga `tracking_forms` activos → sincroniza preguntas del formulario (Graph API) → pagina leads (cursor, 250ms, lookback 90d) → dedup en 3 capas (`external_id` + meta → email → teléfono E.164) → merge o crea lead (`status='new'` , `source='facebook'`).
- **Custom fields:** `meta_lead_id`, `meta_form_id`, `meta_form_name`, `meta_ad/adset/campaign_id` + cada pregunta personalizada.
- **Token:** `META_PAGE_ACCESS_TOKEN` . El "Data Access" caduca cada ~75–90 días. Si caduca → **HTTP 500 + `META_TOKEN_EXPIRED`** dispara alerta. Renovación manual (Graph API Explorer → env var en Vercel).

4 · Modelo de datos (Supabase / Postgres)

Tablas núcleo

Tabla	Para qué	Relaciones clave
users	Cuentas + rol (admin/manager/rep/provider)	→ brands
brands	Marcas/empresas (multi-tenant)	raíz
user_brands	Qué usuario ve qué marca (N:N)	→ users, brands
leads	Prospectos (tel, email, dirección, status, source, assigned_rep_id, external_id/provider)	→ brands, users
calls	Llamadas (dirección, outcome, caller_e164, dialed_e164, tracking_number_id, external_id, recording, transcript)	→ brands, leads, users
tracking_numbers	Números de campaña (phone_e164, provider_metadata.number_id)	→ brands
appointments	Citas (lead→rep→provider, tipo, status, provider_approved, shipped_at)	→ brands, leads, users, clinics
sales	Ventas (amount_cents, payment_status/method, Stripe ids, paid_at)	→ brands, leads, users
sale_items / products	Líneas de venta y catálogo	→ sales, products, brands
payment_plans	Planes a plazos (total, installment_count, frequency_days, first_due_date, overrides)	→ leads, brands
abonos	Pagos individuales de un plan	→ payment_plans, leads
subscriptions	Cobros recurrentes (Stripe)	→ leads, products
tasks	Tareas asignables (priority, status, related_lead_id)	→ users, leads
clinics	Ubicaciones de clínica	→ brands
lead_assignments	Auditoría de reasignaciones	→ leads, users
notifications / notification_prefs	Avisos y preferencias	→ users

Tablas de IA

- **Agente:** agent_runs, agent_actions, agent_pending_actions (cola de aprobación, expira 7d), agent_policies, agent_goals, agent_skills .
- **Conocimiento:** knowledge_patterns, self_reflection_runs, skill_applications .
- **Hermes:** hermes_ticks, hermes_observations, hermes_resolutions .

Idempotencia / índices únicos

- calls UNIQUE (external_id , source) — evita llamadas duplicadas.
- leads_brand_phone_uniq — UNIQUE (brand_id , phone) — evita leads duplicados (creado en esta inspección + 3 duplicados fusionados).

- Manejo de carreras: si choca el teléfono único, se enlaza al "ganador".

Los tipos generados (`src/types/database.ts`) están desactualizados para `caller_e164` , `dialed_e164` , `tracking_number_id` y `external_id/provider` en leads → el código usa `(sb as any)` . Funciona, pero conviene regenerar tipos.

5 · Seguridad y acceso (RLS por rol + marca)

Roles: `admin` (todo) · `manager` (su marca) · `rep` (lo suyo) · `provider` (solo leads de SUS citas).

Recurso	Admin	Manager	Rep	Provider
Leads	todos	de la marca	assigned_rep_id = yo	solo vía appointments
Calls	todas	de la marca	propias	X
Appointments	todas	de la marca	propias	propias
Sales / Payments	todas	de la marca	propias	X
Shipping	✓	✓	X	X
Settings (marcas/usuarios/tracking)	✓	X	X	X

Clientes de Supabase

- **Browser** (anon key) — RLS activa, Client Components.
- **Server** (anon key + cookie) — RLS activa, Server Actions/Components.
- **Admin** (service role) — **bypassa RLS**, solo para operaciones privilegiadas tras verificar el rol.

Auth

- Supabase Auth (password / OTP). Sesión en cookie HttpOnly.
- Middleware redirige a `/login` sin sesión. `PUBLIC_PATHS` : `/api/agent/`, `/api/hermes/`, `/api/webhooks/`, `/api/cron/`, `/auth/confirm`.
- Marca activa en cookie `crm_brand_slug`.

6 · La aplicación (páginas)

Todo bajo `src/app/(app)/`, con sidebar y acceso por rol.

- **/dashboard** — KPIs (llamadas, citas, ventas, pagos), citas de hoy, leads urgentes, insights, resumen del agente. Provider → `/appointments`.
- **/leads** — lista filtrable, export CSV, import (admin/mgr), import-plans, nuevo lead. Provider solo ve leads de sus citas.
- **/leads/[id]** — perfil + timeline (llamadas, citas, ventas), planes con abonos, edición/reasignación.
- **/leads/import-plans** — sube Excel (Plans + Payments), crea leads + planes + abonos.
- **/calls · /calls/[id]** — log (fallback caller_e164), reproductor, transcript, atribución de campaña.
- **/appointments** — citas (lead→rep→provider), estados, aprobación de provider, reasignación inline.
- **/sales (+subscriptions)** — ingresos por venta o por paciente, cobrado vs por cobrar, ticket promedio.
- **/payments-due** — próximos vencimientos (FIFO con overrides), progreso, agregar abono.
- **/shipping** — (admin/mgr) citas aprobadas sin enviar → "Marcar como enviado".
- **/tasks** — tareas por status/prioridad; rep ve las suyas.
- **/settings** — perfil, productos, marca, clínicas, usuarios, tracking numbers.
- **/admin/*** — registro de webhook 800.com, probes, backfills, ECID, insights.

7 · Bot interno (Claude) y Hermes

Bot de consulta — `/api/agent/ask`

- Modelo `claude-sonnet-4-6`, máx 3 rondas de tools, historial de últimos 5 runs.
- Tools de lectura:** `list_brands`, `get_leads`, `get_schedule_today`, `get_sales_kpi`, `get_calls_summary`, `get_tasks_open`.
- Rep-scoping:** admin/manager sin filtro; rep filtrado por `rep_id` / `assigned_rep_id`.
- Zona horaria:** "hoy" se resuelve en Pacific Time, se guarda en UTC.
- Tools de escritura** (`create_task`, `update_lead_status`, `log_call_note`): pasan por autonomía y bandeja de aprobación (`agent_pending_actions`).
- RAG** sobre transcripciones (requiere `OPENAI_API_KEY`). **UI:** sin chat dedicado; botón ☞K + resumen diario + bandeja de pendientes.

Pendiente conocido (#11): algunas tools devuelven `[]` / `{}` cuando la query falla, y el bot puede decir "\$0"/vacío en vez de avisar del error. Cambiarlo toca el contrato de las tools — aún no autorizado.

Hermes — bot de auto-salud (`/api/hermes/tick`)

Ticks periódicos → detecta **observaciones** (duplicados, salud del webhook, leads estancados) con severidad → aplica **resoluciones** automáticas si son seguras, o escala a humano (`requires_approval`).

8 · Crons y disparadores

Vercel (UTC)

Ruta	Horario	Para qué
<code>/api/agent/reflect</code>	07:00	Detección de patrones del bot
<code>/api/agent/daily-insights</code>	07:30	Resúmenes diarios por rep
<code>/api/agent/poll-800com</code>	08:00	Importa llamadas (24h)
<code>/api/agent/transcribe-calls</code>	08:15	Transcribe + embebe audio
<code>/api/agent/poll-meta-leads</code>	08:30	Importa leads de Meta (90d)

cron-job.org (externos)

Ruta	Intervalo	Para qué
<code>/api/agent/poll-meta-leads</code>	15 min	Ingesta casi-realtime de Facebook
<code>/api/cron/auto-link-calls</code>	10 min	Enlaza llamadas huérfanas a leads

9 · Procedimientos operativos

- **Deploy:** `git push` → `npx vercel --prod --yes` → extraer URL → `npx vercel alias set <URL> proyectosagentic-crm.vercel.app` y `... agentic-crm-sigma.vercel.app`. (Rolling Releases NO auto-promueve el alias.)
- **Antes de cada commit:** `npx tsc --noEmit` debe pasar.
- **SQL en Supabase:** se MUESTRA el SQL, el usuario lo corre manualmente. Nunca ejecutar cambios de BD directo.
- **Cambios:** preferir aditivos; verificar tipos; si afecta una operación, revisar y ajustar antes de aplicar.
- **800.com #195485 (A&O)** es la cuenta master; "Si Se Pierde" es uno de los números (3 tracking activos: Buses, Radio, Billboard).

10 · Salud actual y pendientes

Funcionando y verificado OK

- Captura de llamadas (webhook + cron + toast) con atribución de campaña (`dialed_e164` + `tracking_number_id`).
- Identificación en tiempo real del que llama (ECID, gratis-cuando-cacheado).
- Extracción completa de contactos (nombre, dirección, email, carrier) vía conversations + ECID.
- Dedup de leads cerrado en BD (`leads_brand_phone_uniq` + 3 duplicados fusionados).
- Meta Lead Ads activo con detección de token caducado. RLS sólida por rol + marca.

Pendientes PEND

- **#11** — bot que no diga "\$0" cuando una tool falla (cambia contrato de tools, sin autorizar).
- Regenerar tipos de Supabase para quitar los `(sb as any)` .
- Verificar en Vercel: `INTERNAL_CRON_SECRET` y que los jobs de cron-job.org sigan activos.
- Opcional: SQL para mover leads viejos mal asignados al pool.

Para revertir un deploy malo: `npx vercel alias set <deploy-anterior> proyectosagentic-crm.vercel.app` .

Documento generado el 2026-06-08 desde el código real del repositorio `agentic-crm` .